

# Thinker AI

## V1 Architecture Specification

Complete Edition — June 2026

---

Synthesized from Founder Constitution (Extended), Constitution v1.0, and Founding Charter v1.0. This complete edition incorporates all original sections (1–17) together with four architectural addenda: Section 18 (Paid Plan Architecture), Section 19 (Resilience Architecture), Section 20 (Dynamic Routing Engine), and Section 21 (Think Credits & Thinking Levels).

# Table of Contents

---

1. Purpose of This Document
2. V1 Scope Statement
3. Core Philosophy → Implementation Mapping
4. System Architecture ★ REVISED
5. Agent Specifications ★ REVISED
6. Model & Cost Strategy ★ REVISED
7. Data Flow & State ★ REVISED
8. Agent Fallibility & Error Containment ★ REVISED
9. Pipeline Observability & Post-Delivery Feedback
10. Technology Stack
11. Build Order ★ REVISED
12. V1 Launch Checklist
13. Explicitly Out of Scope for V1
14. Open Decisions for Founder Review
15. Goal Intelligence System
16. Output Pipeline — Preview Before Build
17. Multi-Language Support
18. Paid Plan Architecture ★ NEW
19. Resilience Architecture — Continuous Delivery Guarantee ★ NEW
20. Dynamic Routing Engine — Implementation Specification ★ NEW
21. Think Credits, Pricing & Thinking Levels ★ NEW

# 1. Purpose of This Document

---

The three founding constitutions describe Thinker AI's vision and philosophy: a multi-agent system that understands goals, plans work, debates solutions, and delivers verified outcomes. They do not specify how to build it.

This document translates that philosophy into a concrete, buildable Version 1 (V1) architecture. It defines every agent, the data that flows between them, the technology choices, and the order of implementation — scoped deliberately small so V1 can actually ship.

Guiding constraint: all nine agent roles named across the three constitutions are preserved, but each is implemented with the minimum logic needed to fulfil its role. Complexity is added in later versions, not V1. Three additional agents are introduced: Design Agent (Section 5.10), Strategy Agent (Section 5.12), and the expanded Decision Memory system (Section 7.3) — bringing the total to twelve agent roles in V1.

## 2. V1 Scope Statement

---

### 2.1 What V1 Is

A web-based chat application where a user describes a goal (build an app, a website, a game, or a general task), and a dynamic pipeline of twelve lightweight agents — coordinated by Thinker Core — clarifies the request, thinks strategically about it, plans it, builds a single deliverable (including basic image/graphic assets where needed), reviews it, and returns one final answer to the user. Thinker AI supports 100+ languages and responds automatically in the user's language throughout every stage of the pipeline (Section 17).

### 2.2 What V1 Is Not

- Not a fully autonomous multi-day project system (that is a later phase).
- Not a mobile app (V1 ships as a responsive web app).
- Not running every request through full multi-model debate — consensus is invoked selectively (Section 6.4).
- Not retaining credentials, passwords, or payment data in memory.

### 2.3 V1 Success Criteria

| # | Criterion                                                                                 |
|---|-------------------------------------------------------------------------------------------|
| 1 | User can chat normally and get direct answers without triggering the full agent pipeline  |
| 2 | User can request a small project and the system asks clarifying questions before building |
| 3 | All twelve agent roles are present in code, even if some are simple rule-based stubs      |
| 4 | Every final deliverable passes through Reviewer before being shown to the user            |
| 5 | Multi-model consensus works for at least one agent (Judge) end-to-end                     |

| # | Criterion                                                                         |
|---|-----------------------------------------------------------------------------------|
| 6 | Project memory persists within a session and reloads on return                    |
| 7 | For projects with a visual asset, the user explicitly approves the final image    |
| 8 | Smart Clarification correctly detects vague goals and activates 3-Level Deep mode |
| 9 | Signature Question fires on every project-type request before Builder runs        |

### 3. Core Philosophy → Implementation Mapping

| Constitution Principle                                                  | V1 Implementation                                                                              |
|-------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| Understand first, plan second, execute third, verify last               | Fixed pipeline order, enforced in code — but dynamic routing on failure (Section 4.1)          |
| Requirement Clarity Law — ask minimum questions before large work       | Clarification Agent has a hard gate; Builder cannot run until requirements are marked complete |
| Token Economy / Resource Protection Law                                 | Cheap single-model calls by default; multi-model consensus only for high-stakes decisions      |
| One Goal. One Team. One Final Answer.                                   | Thinker Core merges all agent outputs into exactly one response object before returning        |
| No single model/agent/opinion is final — verified consensus is required | Consensus Agent approval is required before Thinker Core marks a deliverable 'final'           |

#### The Founder Law

Thinker AI must not immediately build what the user asks for. It must first determine whether the user's goal, problem, and assumptions are valid. This is the single most important rule that separates Thinker AI from every other AI builder tool. Building first and validating later wastes the user's time, money, and trust. This law is enforced architecturally: the pipeline order makes it structurally impossible to reach Builder without first passing through Clarification Agent and Strategy Agent.

#### The Preview Before Build Law

Thinker AI must show the user what it intends to build before generating the final deliverable. User approval is required at each stage before the next stage begins. A ZIP file that surprises the user is a failure, even if the code is correct. See Section 16 for the full 5-stage output pipeline that implements this law.

## 4. System Architecture ★ REVISED

### 4.1 Pipeline Overview ★ REVISED

The pipeline no longer runs in a fixed 1→12 sequence. Thinker Core operates as a **dynamic routing engine**: after every agent completes, Thinker Core reads the agent's **next\_action** output field and decides the next step. The twelve agent roles are unchanged; what changes is that Thinker Core now reacts to failure and uncertainty dynamically rather than propagating a broken state silently down a fixed chain.

| Condition                               | next_action   | Thinker Core Response                         |
|-----------------------------------------|---------------|-----------------------------------------------|
| Simple chat / general question          | direct_answer | Skip pipeline → single LLM call → return      |
| Request is vague or incomplete          | clarify       | Halt pipeline → return questions to user      |
| Requirements complete, project proposed | proceed       | Run full pipeline in order                    |
| Builder output rejected by Reviewer     | retry         | Re-run Builder (same plan, same step)         |
| Reviewer + Critic both flag deep issue  | explain       | Return to Planner with failure context        |
| Planner→Builder loop exceeds 2 cycles   | clarify       | Return to Clarification for new requirements  |
| Judge score borderline                  | escalate      | Invoke Consensus Agent                        |
| All models for an agent unavailable     | failover      | Model Pool Manager assigns next provider pool |
| Any agent exceeds loop_count limit      | halt          | Pause pipeline, notify user, save state       |

Standard full-pipeline order (when next\_action is 'proceed' throughout): Intent → Clarification → Strategy → Planner → Research → Builder → Reviewer → Critic → Judge → [Consensus if borderline] → Return to user.

### 4.2 Thinker Core Responsibilities ★ REVISED

Thinker Core is not itself an LLM call — it is the orchestration layer (plain backend logic). Original responsibilities retained, plus the following additions:

- Owns the pipeline state machine and dynamic routing table for each conversation.
- Decides whether a message needs the full pipeline or just a direct chat reply.
- Routes each plan step to Builder or Design Agent based on output type (Section 4.3).
- Passes structured data between agents (see Section 7) and enforces the Clarification gate.
- Merges all agent outputs into one final response and writes to memory after each stage.
- **Dynamic next-step decision:** reads next\_action from every agent output before passing control forward.
- **Model Pool Manager:** maintains a pool of available models per agent tier. If a model call fails or times out (5-second threshold), silently reassigns to the next available model in the pool.
- **Loop detection:** tracks loop\_counts per agent. If any agent is retried more than 3 times, or if the Planner→Builder→Reviewer cycle runs more than 2 full times, halts the pipeline and notifies the user.

- **Peer audit trigger:** when a failover model takes over a partially completed task, instructs it to audit what the previous model produced before continuing.
- **State preservation on halt:** on any halt, writes the full Pipeline State to the database before stopping. Resume is always possible from the last completed agent step.

### 4.3 Builder vs. Design Agent Routing

A single project can need both. Routing is decided per step, not per project:

| Planner Step Output Type | Routed To     |
|--------------------------|---------------|
| code, text, config       | Builder Agent |
| image                    | Design Agent  |

If a step's output type is ambiguous, Thinker Core defaults to Builder and lets Builder request an image sub-step from Design Agent if genuinely needed.

### 4.4 Domain Picker — A Context Layer Above the Pipeline

Thinker AI's UI offers named domain agents (Music, DevOps, Security, Coding, etc.) that the user can pick before sending a message. The picker is a domain hint that flows into the one pipeline — it is input, not an alternate path. The twelve agents in Section 5 are the only system that ever produces a deliverable.

| Agent                     | How the domain hint is used                                                                             |
|---------------------------|---------------------------------------------------------------------------------------------------------|
| Intent Agent              | Skipped entirely when a domain is already selected — intent type is derived directly from the domain.   |
| Clarification Agent       | Uses domain-specific required fields where they exist.                                                  |
| Planner Agent             | Receives the domain name in its prompt so step descriptions are framed appropriately.                   |
| Builder Agent             | Receives the domain as additional system-prompt context, biasing toward domain-appropriate conventions. |
| Reviewer / Critic / Judge | Unaffected — quality and completeness checks apply the same way regardless of domain.                   |

Implementation gap (current): the domain picker exists in the mobile UI but the selected domain is not yet included in the request sent to /api/pipeline. Closing this gap is Phase 10 in the build order (Section 11).

## 5. Agent Specifications ★ REVISED

---

All agents now include one additional output field read exclusively by Thinker Core for routing: **next\_action** (proceed | retry | replan | clarify | escalate) and **reason** (string). Agents recommend next\_action but do not execute it — Thinker Core is the sole actor on this field.

### 5.1 Intent Agent

**Role:** Classify the incoming message as chat, app request, website request, game request, or task request. Also outputs a best-guess domain label and the Thinking Level selection (Section 21.3). Skipped entirely when user has already selected a domain via the Domain Picker.

**Input:** Raw user message plus last five messages of context.

**Output:** Intent type, confidence score, domain label, Thinking Level.

**V1 Implementation:** Single LLM call with a classification prompt. No fine-tuning.

**Model Tier:** Fast/cheap.

### 5.2 Clarification Agent ★ UPDATED

**Role:** Enforce the Requirement Clarity Law AND run Smart Clarification — detect emotional context, surface hidden goals, challenge assumptions, and validate the idea before any technical work begins. This is the most critical agent: Wrong Goal → Wrong Strategy → Wrong Plan → Wrong Output.

**Input:** Intent type, full conversation, emotional tone signals.

**Output:** Completeness flag, smart questions, known requirements, assumption flags, Signature Question response.

**V1 Implementation:** LLM call against Smart Clarification Layer + 3-Level Deep Clarification + Constraint & Assumption Discovery + Signature Question Protocol.

**Model Tier:** Fast/cheap with enhanced prompt engineering.

### 5.3 Planner Agent

**Role:** Convert a clarified goal into an ordered list of concrete steps, using the Strategy Agent brief as context.

**Input:** Known requirements object + Strategy Agent brief + Signature Question response + Constraint findings.

**Output:** List of steps, each with id, description, output type, and dependencies.

**V1 Implementation:** Single LLM call producing a structured step list. No re-planning loop in V1.

**Model Tier:** Mid-tier reasoning.

### 5.4 Research Agent

**Role:** Gather external information when a step requires it. Optional — only invoked if Planner flags a step.

**Input:** A single step from the Planner's list, flagged as needing research.

**Output:** Findings and sources.

**V1 Implementation:** Optional. Uses web search. Skipped entirely for purely generative steps.

**Model Tier:** Fast/cheap with search tool access.

## 5.5 Builder Agent

**Role:** Produce the actual deliverable — code, copy, configuration, or content — for each Planner step. Most token-heavy agent.

**Input:** Step description, research findings (if any), relevant project memory.

**Output:** Artifact type, content, generated files.

**V1 Implementation:** Runs once per step, sequentially in V1. Parallel execution is a later optimisation.

**Model Tier:** Strongest available coding/reasoning model.

## 5.6 Reviewer Agent

**Role:** Run the quality control checklist: accuracy, logic, security, completeness, user-goal alignment. For images: checks relevance and appropriateness rather than the code checklist.

**Input:** Builder output plus original requirements.

**Output:** Pass/fail flag and list of issues with severity.

**V1 Implementation:** Single LLM pass against the five-point checklist. If it fails with high-severity issues, the request goes back to Builder once — capped at one retry in V1.

**Model Tier:** Mid-tier reasoning.

## 5.7 Critic Agent

**Role:** Independently look for weaknesses Reviewer's checklist-based pass might miss — edge cases, unclear UX, fragile assumptions. Critic's prompt explicitly withholds Reviewer's verdict until after Critic has formed its own opinion.

**Input:** Builder output plus Reviewer findings.

**Output:** List of concerns with an overall severity rating.

**V1 Implementation:** Single LLM call with adversarial/skeptical framing — deliberately different persona from Reviewer.

**Model Tier:** Mid-tier reasoning.

## 5.8 Judge Agent

**Role:** Score the deliverable on accuracy, feasibility, safety, cost, and completeness, then decide final approval. For high-risk categories (auth, payment, data deletion, permissions), Consensus fires automatically regardless of score.

**Input:** Builder output, Reviewer result, Critic concerns.

**Output:** Score per criterion and approval decision.

**V1 Implementation:** If clearly high or clearly low, Judge decides alone. If borderline, invokes Consensus Agent.

**Model Tier:** Strongest available reasoning model.



## 5.9 Consensus Agent

**Role:** Multi-model consensus used only when Judge needs a tie-breaker or for mandatory high-risk categories. Three personas: Conservative Reviewer, Pragmatic Engineer, Critical Skeptic — never provider names.

**Input:** The same question Judge was uncertain about.

**Output:** Each model's vote and reasoning, plus a final verdict.

**V1 Implementation:** Sends identical question to three different providers in parallel. Majority vote wins; ties default to conservative outcome.

**Model Tier:** Three different providers, called in parallel.

## 5.10 Design Agent

**Role:** Handle visual/graphic output — logos, illustrations, UI mockup imagery, posters, icons. Does not attempt 3D models or animation/video (see Section 13).

**Input:** Planner step flagged as output\_type: image, plus style/brand requirements from Clarification.

**Output:** One or more generated image files plus a short text description.

**V1 Implementation:** Single call to an external image-generation API. Output passes through Reviewer for relevance/appropriateness check.

**Model Tier:** External image-generation provider (e.g. DALL-E or Stability AI).

## 5.11 Visual Asset Clarification & Approval Flow

**Role:** Images cannot be checked against an objective checklist. Reviewer, Critic, and Judge are not equipped to judge taste on the user's behalf, so visual assets follow a different path: (1) Clarification Agent asks one style question before Design Agent runs. (2) Generated image returns to the user with three options: approve, reject and regenerate, or revise. (3) Regeneration is capped at 3 attempts per visual asset in V1.

**Input:** N/A — this is a flow, not a single agent.

**Output:** User approval or revision instruction.

**V1 Implementation:** Approval gate on every generated image before project can be marked complete.

**Model Tier:** N/A.

## 5.12 Strategy Agent

**Role:** Think about the why and how to succeed behind a user's goal — business, product, and market thinking that determines whether the thing being built will actually work. Runs after Clarification, before Planner, on project-type requests only.

**Input:** Clarified requirements + domain hint + Signature Question response + Constraint findings.

**Output:** Structured strategy brief: goal clarity, MVP scope, key risks, success criteria.

**V1 Implementation:** Single LLM call with a structured prompt explicitly asking for business/product thinking. Output is a short brief, not a long document.

**Model Tier:** Mid-tier reasoning.

### 5.2.1 Goal Discovery Mode

When a user's request is too vague to classify or plan, Clarification Agent enters Goal Discovery Mode. Five layers of discovery, one question per layer at a time:

| Layer                 | Purpose                                            | Example Question                                         |
|-----------------------|----------------------------------------------------|----------------------------------------------------------|
| 1. Goal Discovery     | Understand what the user actually wants to achieve | "Why do you want to build this? What is the real goal?"  |
| 2. Problem Discovery  | Identify the real problem being solved             | "What specific problem will this solve?"                 |
| 3. Audience Discovery | Clarify who this is for                            | "Who will use this? What do they currently use instead?" |
| 4. MVP Discovery      | Find the minimum viable starting point             | "If you could only launch with 3 features, which 3?"     |
| 5. Success Discovery  | Define what success looks like                     | "How will you know this worked 90 days after launch?"    |

5.2.2 Smart Clarification Layer

- **Emotional Context Detection:** reads tone, not just words. Softens language and leads with empathy when stress or urgency is detected.
- **Contradiction Detection:** catches conflicts between later requests and earlier stated constraints. Never silently resolves contradictions.
- **Assumption Surfacing:** identifies unstated assumptions and confirms them before Planner locks in scope.
- **Progressive Disclosure:** questions adapt based on previous answers — irrelevant follow-ups are skipped automatically.
- **Intent Confidence Threshold:** 90%+ → proceed; 70–90% → one confirmation question; below 70% → Goal Discovery Mode.

5.2.3 3-Level Deep Clarification

| Level                   | Focus                            | What the Agent Does                                                                         |
|-------------------------|----------------------------------|---------------------------------------------------------------------------------------------|
| Level 1 — Surface Goal  | What did the user literally say? | Captures the stated goal. Confirms understanding before probing deeper.                     |
| Level 2 — Hidden Goal   | Why do they want this?           | Asks why the goal matters. Surfaces the real outcome the user is trying to achieve.         |
| Level 3 — Reality Check | Are their assumptions realistic? | Identifies gaps between stated goal and reality. Presents as a question, never a statement. |

5.2.5 Signature Question Protocol

Every project-type request — without exception — receives exactly one Signature Question before Builder is ever called: **"Why do you believe this is the right solution?"** This single question stops more wrong projects than any amount of technical review. Hard rule: Clarification Agent never blocks a user who insists on proceeding. If the user says 'just build it,' Thinker Core logs the unanswered question, notes the risk in the Strategy Brief, and proceeds. The user's autonomy is always respected.

## 6. Model & Cost Strategy ★ REVISED

### 6.1 Model Tiers ★ REVISED

| Tier                | Used By                         | Purpose                        | Pool Size | Provider Diversity  |
|---------------------|---------------------------------|--------------------------------|-----------|---------------------|
| Fast/cheap          | Intent, Clarification, Research | High-frequency, low-complexity | 6 models  | Min 2 providers     |
| Mid-tier reasoning  | Planner, Strategy, Reviewer     | Structured reasoning           | 3 models  | Min 2 providers     |
| Strongest available | Builder, Judge                  | Highest-stakes generation      | 3 models  | Min 3 providers     |
| Image generation    | Design Agent                    | Visual asset generation        | 2 models  | Min 2 providers     |
| Multi-model         | Consensus Agent                 | Tie-breaking votes             | 3 models  | Exactly 3 providers |

Provider Diversity Rule: within any single agent's model pool, no two models may share the same provider. A provider-wide outage removes at most one model from any pool.

### 6.2 Resource Protection Law in Practice

- Every pipeline run is logged with token cost per agent.
- The Clarification gate prevents Builder, the most expensive agent, from running against an unclear request.
- Consensus Agent is invoked only when Judge's confidence is borderline — estimated at under 20% of all requests.
- The Reviewer retry loop is capped at one retry to prevent runaway cost on a stuck request.
- Think Credits (Section 21) make cost visible and user-controlled — no silent overruns.

### 6.3 Direct Chat Path

Not every message needs the full pipeline. If Intent Agent classifies a message as ordinary chat (next\_action: direct\_answer), Thinker Core skips straight to a single direct-answer call and returns. The full pipeline is reserved for project requests.

### 6.4 When Consensus Actually Fires

| Judge Score                                                    | Action                                            |
|----------------------------------------------------------------|---------------------------------------------------|
| Clearly passes all five criteria                               | Approve directly — no Consensus call              |
| Clearly fails (safety or accuracy critically low)              | Reject directly — send back to Builder            |
| Borderline on any criterion                                    | Invoke Consensus Agent for that criterion only    |
| High-risk category (auth, payment, data deletion, permissions) | Consensus fires automatically regardless of score |

### 6.5 Model Identity Concealment

No provider or model name (Claude, GPT, Gemini, or any other) appears anywhere the user can see — not in chat responses, not in Consensus votes, not in error messages, and not in any debug/admin view shipped in V1. Internally, each model slot is referred to by a generic label (e.g. 'Agent A', 'Agent B'). Failure messages describe failures generically ('one of the verification steps didn't respond') — never naming the provider.

## 6.6 Model Pool Size ★ REVISED

- V1 pool sizes: minimum 3 models per tier for reasoning/generation, minimum 2 for image generation.
- No magic numbers in code — pool size is driven by configuration, not hard-coded constants.
- Per-agent pools, not global: Builder's pool and Reviewer's pool are separate even if they share the same tier.
- Pool health monitoring: a model that fails more than 20% of calls in a rolling 1-hour window is temporarily removed from its pool.
- Beyond V1: pool size becomes fully configurable per agent without a pipeline redesign.

## 7. Data Flow & State ★ REVISED

### 7.1 Pipeline State Object ★ REVISED

Each project in progress is tracked as a single state record. All original fields are retained. The following are added to support dynamic routing and resilience:

| New Field                   | Type        | Purpose                                                                                    |
|-----------------------------|-------------|--------------------------------------------------------------------------------------------|
| current_agent               | string      | Which agent is currently executing                                                         |
| routing_history             | array       | {agent, next_action, reason, model_used, timestamp} — full trace of every routing decision |
| loop_counts                 | object      | {planner: int, builder: int, reviewer: int, ...} — per-agent retry counter                 |
| failover_log                | array       | {agent, failed_model, replacement_model, timestamp, peer_audit_result}                     |
| peer_audit_flag             | boolean     | Set to true when a failover model must audit previous output before continuing             |
| resume_from_agent           | string null | Set on halt — identifies which agent to restart from on resume                             |
| signature_question_response | string null | User's answer to the Signature Question, or null if declined                               |
| constraint_findings         | object      | Output of Constraint Discovery — time, budget, skill, network, existing assets             |
| assumption_flags            | array       | List of surfaced assumptions and whether each was confirmed or flagged as uncertain        |

### 7.2 Memory System

| Memory Type          | Scope                                | V1 Storage                    | Retention Rule                                       |
|----------------------|--------------------------------------|-------------------------------|------------------------------------------------------|
| Short-term           | Single conversation turn             | In-memory / session           | Cleared on session end                               |
| Project memory       | One project across its lifecycle     | Database, keyed by project id | Persists until user deletes the project              |
| Long-term preference | User-level, across projects          | Database, keyed by user id    | Persists indefinitely unless user clears it          |
| Decision Memory      | User-level, explicit saved decisions | Database, keyed by user id    | Persists indefinitely unless user revokes a decision |

Hard rule: passwords, PINs, API keys, and other credentials are never written to any memory layer, even temporarily.

### 7.3 Decision Memory — Detail

Decision Memory is set explicitly — the user makes a deliberate statement like 'always use React for my projects' and expects the system to follow that rule every time without asking again. When a user states a rule explicitly, Thinker Core writes it to Decision Memory and confirms to the user. On every subsequent project, relevant decisions are injected into Planner Agent's context automatically. If a new request conflicts with a saved decision, Thinker Core surfaces the conflict explicitly rather than silently overriding.

## 8. Agent Fallibility & Error Containment ★ REVISED

### 8.1 Why This Section Exists

Every agent is an LLM call, and every LLM call can be wrong. Three distinct failure modes: (1) a single agent gives a wrong verdict, (2) a wrong verdict propagates silently to downstream agents, (3) correlated failure across agents that share the same model family and therefore the same blind spot.

### 8.2 Containment Rules

| Rule                                                                                                                                               | What It Prevents                                                                   |
|----------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| No single agent's approval is final — Judge approval is always required                                                                            | A single fallible verdict reaching the user unchecked.                             |
| If Reviewer and Critic disagree, Thinker Core routes the disputed point to Consensus rather than silently resolved by whichever agent runs next    | Disagreements that it has silently resolved by having the wrong agent run next     |
| Critic's prompt withholds Reviewer's verdict until after Critic has formed its own opinion                                                         | Critic anchoring on 'Reviewer already said it's fine' and skipping its own opinion |
| For high-risk categories (auth, payment, data deletion, permissions), Consensus files a mandatory second review before approving dangerous actions | A shared file is automatically generated by the system, regardless of scale        |
| Every rejection or override is logged with both verdicts and reasoning.                                                                            | Loss of visibility into how often agents actually disagree.                        |

### 8.3 What This Does Not Solve

This containment strategy reduces the chance of a bad deliverable reaching the user undetected — it does not guarantee correctness. Consensus Agent's majority vote can still be wrong if a mistake is common enough across providers. No automated review pipeline replaces a human inspecting high-stakes output, particularly around authentication, payments, or anything touching real user data.

### 8.4 Peer Recovery Protocol ★ NEW

When a model fails mid-task and a replacement model takes over, it follows the Peer Recovery Protocol — it does not simply restart from scratch or blindly continue:

- Step 1 — Failure Detection:** Thinker Core detects failure when a model returns an error, times out after 5 seconds, or returns malformed output. Detection is automatic.
- Step 2 — Context Handoff:** Thinker Core passes the full Pipeline State to the replacement model, including the original agent input, any partial output, and peer\_audit\_flag set to true.
- Step 3 — Peer Audit:** The replacement model first reviews what the failed model produced. It identifies what is correct, what is incorrect, and what is missing. Audit result is written to failover\_log.
- Step 4 — Targeted Continuation:** The replacement model generates only what is needed — keeps correct partial output, fixes incorrect output, completes missing output. Never regenerates correct work.
- Step 5 — Normal Pipeline Resumes:** The replacement model's output is treated identically to a first-attempt output and proceeds through Reviewer, Critic, and Judge as normal.

### 8.5 Loop Detection Rules ★ NEW

| Loop Type                               | Threshold     | Thinker Core Response                                                 |
|-----------------------------------------|---------------|-----------------------------------------------------------------------|
| Single agent retry (same step)          | 3 attempts    | Halt pipeline. Save state. Notify user.                               |
| Planner → Builder → Reviewer full cycle | 2 full cycles | Return to Clarification with failure context. Ask user to re-scope.   |
| Clarification loop (vague answers)      | 4 rounds      | Surface the specific unknown directly.                                |
| Judge → Consensus → reject cycle        | 2 cycles      | Halt and present best available output with explicit quality caveats. |

## 9. Pipeline Observability & Post-Delivery Feedback

### 9.1 The Silent Failure Problem

When a user says 'this output is bad,' the problem could have originated at any point in the pipeline. Because agents run sequentially and each feeds the next, a mistake at step 1 propagates silently through all subsequent steps. Without observability, even the founder cannot debug what went wrong.

### 9.2 Per-Agent Logging (Founder-Visible)

| Field                | What It Records                                                                     |
|----------------------|-------------------------------------------------------------------------------------|
| agent_name           | Which agent ran                                                                     |
| input_summary        | Short description of what this agent received as input                              |
| output_summary       | Short description of what this agent produced                                       |
| duration_ms          | How long the agent took in milliseconds                                             |
| status               | success, failed, or skipped                                                         |
| error_detail         | If status is failed — error message using generic terms, never model/provider names |
| confidence           | Where applicable — Judge's score, Consensus vote breakdown, Reviewer pass/fail      |
| retry_count          | How many times this agent was retried before succeeding or giving up                |
| failover_cost        | Token spend from failover calls (separate from normal pipeline cost)                |
| clarification_depth  | Which Clarification layers ran (Clarification Agent only)                           |
| signature_q_answered | Whether the user answered the Signature Question (Clarification Agent only)         |

### 9.4 Post-Delivery Feedback Loop

| User's Problem                | System Response                                                                                           |
|-------------------------------|-----------------------------------------------------------------------------------------------------------|
| Bug in the output             | Ask user to describe the bug, send back to Builder with bug description + original plan. Reviewer re-r    |
| Missing feature               | Treat as a new small requirement. Run Clarification → Planner (for just the missing piece) → Builder      |
| Style/design not to taste     | Follow Visual Asset revision flow (Section 5.11) for images; ask for specific style feedback for code/la  |
| Vague ('it's bad', no detail) | Ask one clarifying question: 'Is there a specific feature that's broken, something missing, or does it lo |

### 9.5 Fix Limits — Cost-Based, Not Count-Based

| Fix Type                                            | What Runs                   | Limit                        |
|-----------------------------------------------------|-----------------------------|------------------------------|
| Small targeted fix (change a color, update a label) | Builder only                | Unlimited                    |
| Medium fix (fix a bug, add a missing feature)       | Builder + Reviewer + Critic | Up to 10 per project session |



| Fix Type                                    | What Runs                            | Limit                       |
|---------------------------------------------|--------------------------------------|-----------------------------|
| Full rebuild (start over from requirements) | Full pipeline from Planner onwards   | Up to 3 per project session |
| Rollback (restore a previous version)       | Database lookup only — no agent runs | No limit — always free      |

## 9.6 Version History

Every time Builder produces an output, Thinker Core saves that version to the database before overwriting the current state. Only the last 10 versions are kept per project in V1. Rollback restores a previous version instantly with no rebuild — rollbacks never count against any fix limit.

## 9.7 Selective Rollback & Merge

A full rollback restores everything. A selective rollback restores a previous version as the base while keeping specific elements from the current version. Five rules: (1) Full Rollback = database lookup only. (2) Selective Rollback = Clarification clarifies what to keep → previous version restored → Builder applies only selected elements → Reviewer checks merged result. (3) Rollback never destroys history. (4) Every selective rollback creates a new version entry. (5) User-selected changes always override rollback state — Thinker Core never silently resolves a merge conflict.

## 10. Technology Stack (V1 Recommendation)

| Layer      | Choice                                                        | Reasoning                                                               |
|------------|---------------------------------------------------------------|-------------------------------------------------------------------------|
| Frontend   | React + Vite                                                  | Matches existing developer experience                                   |
| Backend    | Node.js (Express) or Firebase Cloud Functions                 | Lightweight, matches existing Firebase familiarity                      |
| Database   | Firestore                                                     | Document model fits the pipeline state object well                      |
| Auth       | Firebase Auth                                                 | Reuse existing pattern                                                  |
| LLM Access | Direct API calls to each provider, made only from the Backend | Protect the API keys, matches Resource Protection Law                   |
| Payments   | Stripe                                                        | Webhook-based plan tier updates; Thinker AI never touches raw card data |
| Hosting    | Firebase Hosting or Vercel                                    | Simple, matches existing deployment habits                              |

## 11. Build Order (Implementation Roadmap) ★ REVISED

| Phase            | Deliverable                                                                                                                                               | Status    |
|------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|
| Phase 1          | Thinker Core skeleton + direct chat path (passthrough)                                                                                                    | Unchanged |
| Phase 1.5 ★ NEW  | Dynamic Routing Engine: routing table, next_action handling, loop_counts, loop_detection, loop_protection, loop_recovery, loop_protection (Section 5.2.1) | Unchanged |
| Phase 1.6 ★ NEW  | Model Pool Manager — failover detection (5s timeout), pool health monitoring, New-Router-Complete (Section 5.2.2)                                         | Unchanged |
| Phase 2          | Intent Agent + Clarification Agent (basic) + hard gate                                                                                                    | Unchanged |
| Phase 2.5        | Smart Clarification Layer (Section 5.2.2)                                                                                                                 | Unchanged |
| Phase 2.6        | 3-Level Deep Clarification + Signature Question (Sections 5.2.3, 5.2.5)                                                                                   | Unchanged |
| Phase 2.7        | Constraint Discovery + Assumption Discovery (Section 5.2.4)                                                                                               | Unchanged |
| Phase 3          | Planner Agent + Builder Agent (single-step)                                                                                                               | Unchanged |
| Phase 4          | Reviewer Agent + one-retry loop back to Builder                                                                                                           | Unchanged |
| Phase 5          | Critic Agent + Judge Agent (Judge decides alone)                                                                                                          | Unchanged |
| Phase 6          | Consensus Agent + borderline-routing from Judge                                                                                                           | Unchanged |
| Phase 7          | Research Agent (optional step augmentation)                                                                                                               | Unchanged |
| Phase 8          | Design Agent + Builder/Design routing (Section 4.3)                                                                                                       | Unchanged |
| Phase 8.5        | Strategy Agent (Section 5.12)                                                                                                                             | Unchanged |
| Phase 9          | Project memory + long-term preference + Decision Memory                                                                                                   | Unchanged |
| Phase 10         | Domain picker wiring (Section 4.4)                                                                                                                        | Unchanged |
| Phase 10.5       | Per-agent logging (Section 9.2) + post-delivery feedback (Section 9.4)                                                                                    | Unchanged |
| Phase 10.6 ★ NEW | Resilience observability: failover_log dashboard, pool health alerts, routing_history trace viewer                                                        | Unchanged |
| Phase 10.7 ★ NEW | Think Credits integration: credit deduction engine, balance checks, confirmation prompts (Section 21)                                                     | Unchanged |
| Phase 11         | Polish: UI, error states, retry UX, and the launch checklist (Section 12)                                                                                 | Unchanged |

## 12. V1 Launch Checklist

---

- Stable chat experience: the direct path works without errors
- Full agent pipeline completes end-to-end on at least three different project types (app, website, game)
- Strategy Agent produces a meaningful strategic brief, visibly reflected in Planner's output
- Decision Memory saves an explicit user rule, applies it automatically, and surfaces conflicts correctly
- Design Agent produces a usable image; approve / reject / revise all work correctly; 3-attempt cap enforced
- Clarification gate cannot be bypassed
- Smart Clarification correctly detects emotional tone and adjusts question framing
- Contradiction Detection catches at least one case where user's later message contradicts an earlier constraint
- Signature Question fires on every project-type request; response stored in Pipeline State
- Intent Confidence Threshold correctly routes: high → proceed, mid → confirm, low → Goal Discovery Mode
- No credentials ever appear in stored memory (manual audit)
- Consensus Agent tested with intentionally borderline cases
- High-risk categories tested to confirm Consensus fires automatically
- Token cost per request is logged and reviewed
- Final response is always a single merged answer, never raw agent dumps
- 5-stage output pipeline works end-to-end (Section 16)
- Live Preview renders correctly for web projects
- Language detection works: user writes in any Tier 1 language and all agent responses are in that language
- RTL language (Arabic or Hebrew) renders correctly
- Per-agent log entries written for every pipeline run; failed run traceable to specific agent
- Cost-based fix limits enforced: small unlimited, medium capped at 10, full rebuild capped at 3
- Version history saves correctly; rollback restores without Builder call; 10-version limit enforced
- Selective rollback works correctly
- Domain picker measurably changes pipeline output
- No provider/model names reachable from any user-facing surface including error states
- Dynamic routing engine tested: all next\_action values handled correctly
- Failover tested: model timeout triggers pool reassignment within 5 seconds
- Peer Recovery Protocol tested: replacement model audits and continues partial work correctly
- Loop detection tested: exceeding thresholds triggers halt and state save
- Think Credits deducted correctly per action; no silent overruns; confirmation shown above 3 credits

- Stripe webhook correctly updates plan tier on payment and downgrade
- Free Trial 50-credit one-time limit enforced; no monthly refill

## 13. Explicitly Out of Scope for V1

---

- Parallel multi-step Builder execution
- Automatic re-planning when a step fails repeatedly (Peer Recovery handles model-level failure; plan-level re-planning is V2)
- Native mobile app (Play Store launch can wrap the web app initially)
- Organisational/team-shared memory
- Autonomous multi-day project system with no human checkpoint
- Native Thinker foundation model (V1 uses third-party model APIs only)
- 3D model generation
- Animation and video generation
- UI/UX design as a distinct discipline — Design Agent produces static images only, not Figma-format output

## 14. Open Decisions for Founder Review

---

1. Consensus uses exactly three providers — confirmed for V1, configurable beyond three in later versions.
2. Reviewer retry is capped at one loop — confirm this is acceptable or should be configurable.
3. Research Agent is optional and skippable per step — confirm this matches intended behaviour.
4. Technology stack assumes continuity with existing Firebase-based project — confirm versus fresh stack.
5. High-risk category list (auth, payments, data deletion, permissions) — confirm this list is complete.
6. Design Agent's image provider not yet chosen between DALL-E or Stability AI — confirm based on cost, quality, and licensing.
7. Free Trial credit amount set at 50 (one-time) — confirm this is the right conversion balance.
8. Pro price set at \$19/month and Founder at \$59/month — confirm based on competitive positioning.

## 15. Goal Intelligence System

---

This section brings together the highest-level thinking layer of Thinker AI — the system that determines whether a user's goal is valid, well-formed, and worth building, before any technical work begins. This is what makes Thinker AI a thinker, not just a builder.

### 15.1 Goal Discovery

Goal Discovery is the process of understanding what the user actually wants to achieve. When a user's goal is unclear, Goal Discovery Mode activates (Section 5.2.1) and works through five layers of questions — one at a time, conversationally. Goal Discovery is not an obstacle — it is the product. A user who spends 5 minutes in Goal Discovery before building is far more likely to end up with something useful.

### 15.2 Problem Discovery

Problem Discovery is focused specifically on identifying the real problem being solved. If the problem is not real, or not specific enough, no amount of good engineering will produce a successful product. Thinker AI asks: 'What specific problem will this project solve?' and follows up until the answer is specific.

### 15.3 Idea Validation

After Goal and Problem Discovery, Strategy Agent runs an Idea Validation covering four questions: (1) Is the problem real? (2) Is this idea the right solution? (3) Is this the right time? (4) Why this team? The output is not a go/no-go decision — it is a brief that sharpens the plan Planner receives.

### 15.5 Founder Mode

Activated when a user is thinking through an entire product, business, or startup idea. Can be triggered explicitly ('Activate Founder Mode') or detected automatically when the request involves market, revenue, competition, or business model language. In Founder Mode, Strategy Agent expands its scope to cover: Problem → Market → Competition → Business model, full risk landscape, and a Go/Caution/No-Go assessment (always advisory, never a block).

## 16. Output Pipeline — Preview Before Build

Instead of building everything silently and delivering a ZIP at the end, Thinker AI follows a 5-stage pipeline where the user sees and approves each stage before the next begins. This implements the Preview Before Build Law (Section 3).

| Stage | Name                 | What User Sees                                                             | User Action                                                 |
|-------|----------------------|----------------------------------------------------------------------------|-------------------------------------------------------------|
| 1     | Thinking Summary     | Goal understanding, strategy brief, proposed approach                      | Confirm or correct before anything is built                 |
| 2     | Blueprint Preview    | Full plan: screens/components, tech stack, estimated app complexity        | Approve, remove/add items, or request changes               |
| 3     | Build Preview (live) | Real-time progress + live rendered UI updating as each screen is completed | Approve first completed mandatory, or change directly       |
| 4     | Approval             | Full app visible in preview panel                                          | Approve, change something specific, or go back to blueprint |
| 5     | Final Output         | Project Summary, Live Preview Link, ZIP download, Download and Notes       |                                                             |

Simple requests skip stages 2 and 4. The user is never surprised by what was built — even for simple outputs, Thinker Core shows a one-line summary and waits for brief confirmation before running Builder.



## 17. Multi-Language Support

### 17.1 Core Principle

Thinker AI detects the user's language automatically from their first message and responds in that same language throughout the entire session — all agent responses, clarification questions, strategy briefs, progress indicators, error messages, and final summaries. The user never has to specify their language or change a setting.

### 17.2 What Gets Translated vs. What Stays in English

| Content Type                                            | Language Used                                       | Reason                                               |
|---------------------------------------------------------|-----------------------------------------------------|------------------------------------------------------|
| All conversation with user                              | User's language                                     | This is the product interface                        |
| Clarification questions, strategy brief, User's Display | User's language                                     | User must understand these clearly                   |
| Progress indicators, error messages                     | User's language                                     | UX must be native-feeling                            |
| Generated code (variables, functions, classes)          | English                                             | Industry standard; must be readable by any developer |
| Code comments                                           | User's language (if requested) or English (default) | English (default) via Decision Memory                |
| README and documentation                                | User's language                                     | User needs to understand their own project           |
| Folder/file names, API endpoint names                   | English                                             | Compatibility and portability                        |

### 17.4 Supported Language Tiers

| Tier                   | Languages                                                                                                                                          | Quality in V1                                    |
|------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------|
| Tier 1 — Full support  | English, French, Spanish, German, Portuguese, Italian, Dutch, Japanese, Korean, Chinese (Simplified), Arabic, Hebrew, Urdu, Farsi, and others      | High quality                                     |
| Tier 2 — Good support  | 50+ additional languages including Urdu, Malay, Thai, Swahili, Czech, Slovak, Norwegian, Danish, Greek, Polish, Vietnamese, Indonesian, and others | Good quality                                     |
| Tier 3 — Basic support | Remaining languages supported by the underlying model provider                                                                                     | Response quality depends on the model's training |

RTL Languages (Arabic, Hebrew, Urdu, Farsi, and others): fully supported in all user-facing text. The UI must render these correctly — right-aligned text, mirrored layout where appropriate. V1 must be tested with at least one RTL language before launch.

## 18. Paid Plan Architecture ★ NEW

All plan enforcement happens inside Thinker Core, immediately after the incoming request is received and before any agent is invoked. No billing or quota logic exists inside individual agents.

### 18.1 Plan Tiers Overview

| Tier       | Price                    | Think Credits                | Target User                       |
|------------|--------------------------|------------------------------|-----------------------------------|
| Free Trial | \$0                      | 50 (one-time, never refills) | Individuals exploring the product |
| Pro        | \$19/month or \$199/year | 1,500/month                  | Freelancers, indie builders       |
| Founder    | \$59/month or \$589/year | 5,000/month                  | Power users, startup founders     |
| Enterprise | Custom                   | Custom                       | Organisations 10+ seats (V2)      |

### 18.2 Feature Gate Matrix

| Feature                                         | Free Trial            | Pro            | Founder        |
|-------------------------------------------------|-----------------------|----------------|----------------|
| Direct Chat                                     | ✓                     | ✓              | ✓              |
| Intent Agent                                    | ✓                     | ✓              | ✓              |
| Basic Clarification                             | ✓                     | ✓              | ✓              |
| Smart Clarification (3-Level Deep, Signature Q) | —                     | ✓              | ✓              |
| Strategy Agent                                  | —                     | ✓              | ✓              |
| Founder Mode                                    | —                     | ✓              | ✓              |
| Planner Agent                                   | ✓                     | ✓              | ✓              |
| Research Agent                                  | —                     | Limited        | Unlimited      |
| Builder Agent                                   | Credit-limited        | Credit-limited | Credit-limited |
| Reviewer + Critic + Judge                       | Reviewer + Judge only | ✓              | ✓              |
| Consensus Agent                                 | —                     | —              | ✓              |
| Design Agent                                    | —                     | ✓              | ✓              |
| 5-Stage Output Pipeline                         | Stage 1+5 only        | Full           | Full           |
| Version History & Rollback                      | 3 versions            | 10 versions    | 25 versions    |
| Decision Memory                                 | —                     | —              | ✓              |
| Thinking Level: High (■)                        | —                     | ✓              | ✓              |
| Thinking Level: Consensus (■)                   | —                     | —              | ✓              |
| Priority Queue                                  | —                     | —              | ✓              |

| Feature           | Free Trial    | Pro           | Founder       |
|-------------------|---------------|---------------|---------------|
| Extra credit rate | \$0.10/credit | \$0.07/credit | \$0.05/credit |

## 18.3 Enforcement Architecture

Thinker Core enforces plan rules in 5 steps before any agent runs:

**Step 1 — Authenticate:** Read user's plan tier from session token. Plan tier is refreshed on billing events, never re-fetched mid-pipeline.

**Step 2 — Feature gate check:** Check whether the requested pipeline path is permitted for this tier. A gated feature returns a structured error with upgrade prompt — the pipeline does not partially execute.

**Step 3 — Credit balance check:** Read Think Credit balance. If insufficient for estimated session cost, block and show top-up or downgrade options.

**Step 4 — Concurrent run check:** If user already has the maximum active pipeline runs for their tier, queue the new request rather than rejecting it.

**Step 5 — Pipeline proceeds:** If all checks pass, pipeline runs normally. Plan tier is injected into Pipeline State for observability logging only.

## 18.4 Billing Integration

- Stripe Checkout handles all payment collection. Thinker AI never touches raw card data.
- On successful payment, Stripe sends a webhook to a dedicated billing endpoint (not /api/pipeline). The billing endpoint updates the user's plan\_tier in Firestore and refreshes the session token.
- Downgrade takes effect at end of current billing period. User retains access until then. On downgrade, Pro-created projects are preserved but gated features become read-only.
- Failed payment: user enters 'grace' state — 7 days of continued access while Stripe retries. After 7 days without resolution, tier reverts to Free Trial automatically.
- Refunds handled entirely in Stripe. The plan\_tier change is the only signal Thinker Core needs.

# 19. Resilience Architecture — Continuous Delivery Guarantee

★ NEW

The Continuous Delivery Guarantee: Thinker AI never holds a user's work hostage to a single model, a single provider, or a temporary outage. When any model fails, another model takes the task, audits what was done, and continues. The user's work always moves forward.

## 19.1 No Single Point of Failure

- Minimum pool size: 3 models for all reasoning and generation agents.
- Provider diversity: no two models in the same agent's pool may share a provider.
- Pool configuration is static per deployment — Thinker Core takes a pool\_health\_snapshot at pipeline start.
- If a pool's health snapshot shows fewer than 2 available models at pipeline start, Thinker Core notifies the operator (not the user) and proceeds with reduced redundancy.

## 19.2 Failure Detection Timeline

| Time                           | Event                            | Thinker Core Action                                |
|--------------------------------|----------------------------------|----------------------------------------------------|
| 0s                             | Model call initiated             | Start 5-second timeout clock                       |
| 0–5s                           | Model responds normally          | Continue pipeline — no action needed               |
| 5s                             | No response received             | Mark model as failed. Assign next model from pool. |
| 5–7s                           | Replacement model initialises    | Peer audit begins (Section 8.4 Steps 2–3)          |
| 7s+                            | Replacement model continues task | Pipeline resumes — user sees no pause              |
| All pool models fail exhausted |                                  | Loop detection triggers halt. State saved.         |

## 19.3 User Communication During Failures

| Situation                          | Message to User                                                                                                                        |
|------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| Failover resolves within 7s        | No message. Pipeline continues normally.                                                                                               |
| Failover takes 7–30s               | 'Still working on this — taking a moment longer than usual.'                                                                           |
| Partial completion, halt triggered | 'I've completed [X of Y steps]. I ran into a difficulty on the next step — your work so far is saved. Would you like to try again?'    |
| All models unavailable             | 'I wasn't able to complete this step right now — the service I need isn't responding. Your work is saved. I'll try again in a moment.' |

Hard rule: no provider or model name appears in any user-facing message, including failure messages (Section 6.5).

## 19.4 Failover Cost Rules

- Failover additional model calls are absorbed by the operator — never charged to the user (0 credits).
- Peer audit scope is minimal — replacement model audits only the failed model's partial output.
- Targeted continuation — replacement generates only what is missing or incorrect; correct partial output is reused.
- Failover token spend is logged under `failover_cost` in the per-agent log (Section 9.2), separate from normal pipeline cost.
- A model failing more than 20% of calls in a rolling 1-hour window is temporarily removed from its pool.

## 20. Dynamic Routing Engine — Implementation Specification

### ★ NEW

---

This section specifies the internal design of the routing layer inside Thinker Core introduced in Section 4.1. It is an implementation guide for the developer, not a user-facing feature description.

### 20.1 Routing Engine Responsibilities

- Read `next_action` from every agent output before passing control forward.
- Maintain the `routing_history` array in Pipeline State — one entry per routing decision.
- Enforce `loop_count` thresholds (Section 8.5) and trigger halt when exceeded.
- Manage the Model Pool Manager: track pool health, execute failovers, trigger peer audits.
- Never expose routing internals to agents — agents receive only their defined inputs.
- Never expose routing internals to users — users see only the result.

### 20.2 Routing Decision Logic (pseudocode)

```
function routeAfterAgent(agent_output, pipeline_state):  
    action = agent_output.next_action  
    log_routing_decision(agent_output, pipeline_state)  
  
    if action == 'proceed':  
        return dispatch(next_in_sequence(pipeline_state.current_agent), pipeline_state)  
    if action == 'retry':  
        pipeline_state.loop_counts[current_agent] += 1  
        if loop_counts[current_agent] > 3: return trigger_halt(pipeline_state)  
        return dispatch(current_agent, pipeline_state)  
    if action == 'replan':  
        pipeline_state.loop_counts['planner_cycle'] += 1  
        if loop_counts['planner_cycle'] > 2: return dispatch('clarification', pipeline_state)  
        return dispatch('planner', pipeline_state_with_failure_context)  
    if action == 'clarify': return dispatch('clarification', pipeline_state)  
    if action == 'escalate': return dispatch('consensus', pipeline_state)  
    if action == 'direct_answer': return dispatch('direct_llm_call', pipeline_state)
```

### 20.3 Model Pool Manager Logic (pseudocode)

```
function callAgentWithFailover(agent_name, input, pipeline_state):  
    pool = get_pool(agent_name) # from pool_health_snapshot  
    for model in pool.available_models:  
        try:
```

```
result = call_model(model, input, timeout=5s)
if is_valid_output(result): return result
except TimeoutError, ModelError:
log_failover(agent_name, model, pipeline_state)
next_model = pool.next_available(exclude=failed_models)
if next_model:
input_with_audit = add_peer_audit_flag(input, partial_result)
# recurse with updated exclusion list
else:
return trigger_halt(pipeline_state) # pool exhausted
```

## 20.4 What the Routing Engine Is Not

- Not an LLM call — the routing engine is plain backend logic (conditional branching on next\_action values).
- Not visible to agents — agents receive their defined inputs and return their defined outputs.
- Not a replacement for good agent design — it handles failures gracefully, not poor prompts.
- Not a billing layer — plan tier checks (Section 18.3) run before the routing engine, not inside it.

## 21. Think Credits, Pricing & Thinking Levels ★ NEW

### The Thinker AI Golden Rule

**More Thinking**  
**= More Questions**  
**= More Validation**  
**= More Credits**

Users do not pay for answers. They pay for the depth of thinking behind the answer. This principle aligns every pricing tier, every credit cost, and every agent behaviour in Thinker AI into a single coherent system.

### 21.1 Why Every Interaction Costs Credits

Consumer AI products like ChatGPT or Claude.ai own their foundation models — their marginal cost per message approaches zero. Thinker AI is architecturally different: every agent call invokes third-party model APIs (Section 6.1). Each call has a real monetary cost paid to the provider. The Think Credit system makes this cost visible, user-controlled, and sustainable — every interaction, including simple chat, costs at least 1 credit.

### 21.2 Thinking Levels

| Level              | Icon | Purpose                                | Agents Active                                          | Base Cost  |
|--------------------|------|----------------------------------------|--------------------------------------------------------|------------|
| Low Thinking       | ■    | Fast answers, simple chat              | Intent + Direct Answer only                            | 1 credit   |
| Medium Thinking    | ■    | Analysis, comparisons, recommendations | Intention Clarification + Planner + Builder + Reviewer | 8 credits  |
| High Thinking      | ■    | Deep planning, full project builds     | Full pipeline (all 12 agents)                          | 10 credits |
| Consensus Thinking | ■    | Multi-model validation, startup ideas  | Full pipeline + Consensus (3 providers)                | 30 credits |

Thinker Core auto-selects the Thinking Level during Intent Agent classification. The user can accept, downgrade, or upgrade before the pipeline runs. Credits are never deducted without user confirmation for sessions costing more than 1 credit.

### 21.3 Plan Restrictions by Thinking Level

| Thinking Level       | Free Trial | Pro | Founder |
|----------------------|------------|-----|---------|
| ■ Low Thinking       | ✓          | ✓   | ✓       |
| ■ Medium Thinking    | ✓          | ✓   | ✓       |
| ■ High Thinking      | —          | ✓   | ✓       |
| ■ Consensus Thinking | —          | —   | ✓       |



## 21.4 Clarification & Goal Discovery Depth by Thinking Level

| Layer                 | ■ Low          | ■ Medium      | ■ High        | ■ Consensus                |
|-----------------------|----------------|---------------|---------------|----------------------------|
| Clarification style   | Direct (0–2 Q) | Smart (3–5 Q) | Deep (5–10 Q) | Deep + Challenge (Dynamic) |
| Goal Discovery        | —              | Light         | Full          | Full                       |
| Problem Discovery     | —              | Light         | Full          | Full                       |
| Audience Discovery    | —              | Light         | Full          | Full                       |
| MVP Discovery         | —              | Optional      | Full          | Full                       |
| Success Discovery     | —              | Optional      | Full          | Full                       |
| Assumption Discovery  | —              | —             | Conditional   | Full                       |
| Constraint Discovery  | —              | Optional      | Full          | Full                       |
| Alternative Discovery | —              | —             | Conditional   | Full                       |
| Reality Challenge     | —              | —             | Light         | Strong                     |
| Founder Mode          | —              | —             | —             | ✓                          |

## 21.5 Credit Cost Table

| Action                                   | Credits | Notes                                                |
|------------------------------------------|---------|------------------------------------------------------|
| Simple chat / direct answer              | 1       | Per message. Always applies.                         |
| Clarification reply (per round)          | 2       | Each round of questions + user answer = 2 credits    |
| Research Agent (per step)                | 3       | Only when Planner flags a step as research-needed    |
| Medium analysis                          | 4       | Medium Thinking sessions                             |
| Image generation (per image)             | 10      | Per image, per attempt — including revision attempts |
| High Thinking session (base)             | 10      | Charged once at session start                        |
| Consensus session (base)                 | 30      | Charged once at session start                        |
| Small fix (Builder only)                 | 1       | Color change, label update, minor edit               |
| Medium fix (Builder + Reviewer + Critic) | 5       | Bug fix, missing feature                             |
| Full rebuild (Planner onwards)           | 50      | Complete restart from planning                       |
| Rollback (any version)                   | 0       | No LLM call — database lookup only                   |
| Selective rollback + merge               | 3       | One small Builder call + one Reviewer call           |
| Failover (model substitution)            | 0       | Absorbed by operator — never charged to user         |

Confirmation rule: for any single action costing more than 3 credits, Thinker Core shows the cost before executing and waits for user confirmation.

21.6 Worked Examples

| User Request                              | Level      | Flow                                                                                               | Total Cost                           |
|-------------------------------------------|------------|----------------------------------------------------------------------------------------------------|--------------------------------------|
| React ■■■?                                | ■ Low      | Intent → Direct Answer                                                                             | 1 credit                             |
| React vs Vue — ■■■■ Startup ■■■■? ■■■■    | ■■■ Medium | Intent → Clarification (2 Q) → Planner → Builder → Reviewer                                        | 3 + 2 + 4 = 9 credits                |
| Food Delivery Startup ■■■■ ■■■■ ■■■■      | ■■■ Medium | Intent → Clarification (3 rounds, Deep Discovery) → Strategy → Planner → Builder → Reviewer        | 15 + 5 + 5 + 5 + 5 = 35 credits      |
| ■■■■ Startup Idea validate ■■■■ Consensus | ■■■ Medium | Intent → Clarification (5 rounds, all layers) → Strategy → Planner → Builder → Reviewer → Pipeline | 90 + 10 + 10 + 25 + 75 = 220 credits |

21.7 Credit Enforcement Rules

- **Check before dispatch:** Thinker Core checks credit balance before dispatching any agent. Insufficient balance = pipeline does not start.
- **Confirm above 3 credits:** any single action costing more than 3 credits shows: action name, credit cost, current balance, balance after. User must confirm.
- **Deduct on completion, not on start:** credits deducted when an agent successfully completes, not when dispatched. Failed calls that trigger failover are not charged.
- **Failover is always free:** model substitution events never consume user credits.
- **Partial session crediting:** if pipeline halts mid-run, credits charged only for agents that successfully completed. Thinking Level base cost still applies if session was initialised.
- **Low balance warning:** balance below 20 credits shows a persistent indicator. Balance at 0 shows top-up prompt before any new session.
- **No silent overruns:** if mid-session estimated remaining cost exceeds remaining balance, Thinker Core pauses and asks the user to confirm or stop.

The Thinker AI Golden Rule

More Thinking  
= More Questions  
= More Validation  
= More Credits

Every credit a user spends is a unit of reasoning, validation, and challenge applied to their goal. The more they invest in thinking, the less likely they are to build the wrong thing. This is Thinker AI's value proposition — and its pricing model.



# Thinker AI V1 Architecture Spec

## Addendum 3 — Sections 22 & 23

### File Upload Architecture | UI/UX Specification

June 2026

---

Section 22 defines how Thinker AI handles every type of file a user can upload — documents, code, images, data files, and ZIP archives — including a new Document Agent (the 13th agent role in V1). Section 23 specifies the complete UI/UX structure: every screen, every settings panel, every profile option, and every in-chat control a user can interact with.

## 22. File Upload Architecture

Thinker AI accepts any file a user uploads. Thinker Core identifies the file type, routes it to the correct agent, and determines what to do with it — the user does not need to specify 'review this code' or 'analyze this PDF.' Thinker reads the file and asks the right question.

### 22.1 Core Principle

**File Upload Pipeline:** File arrives → Thinker Core identifies type → Security scan → Route to correct agent → Agent reads & understands → Clarification Agent asks what to do → Pipeline runs → Deliver result.

### 22.2 Supported File Types & Routing

| Category   | Formats                                                   | Max Size | Routed To                                             | Credit Cost                                                       |
|------------|-----------------------------------------------------------|----------|-------------------------------------------------------|-------------------------------------------------------------------|
| Documents  | PDF, DOCX, TXT, MD                                        | 50 MB    | Document Agent (Section 22.7)                         | 3 credits per file                                                |
| Code files | JS, TS, PY, HTML, CSS, JSON, XML, and all code extensions | 10 MB    | Review extension → Judge → Builder (if fix requested) | 5 credits review<br>10 credits fix                                |
| Images     | PNG, JPG, JPEG, SVG, WEBP, GIF                            | 20 MB    | Design Agent                                          | 2 credits analysis<br>15 credits edit<br>20 credits reference-gen |
| Data files | CSV, XLSX, JSON                                           | 25 MB    | Research Agent                                        | 3 credits per file                                                |
| Archives   | ZIP, TAR                                                  | 100 MB   | Thinker Core (extract & re-route each file)           | 5 credits extract<br>+ per-file costs                             |

Blocked file types: .exe, .bat, .sh, .dll, .bin, .dmg, and any executable format. These are rejected before processing begins — no credits are charged.

### 22.3 ZIP / Archive Upload Flow

A ZIP upload is treated as a full project hand-off. Thinker Core extracts the archive, maps the structure, and then asks the user what to do — rather than assuming.

- ZIP arrives → Thinker Core extracts to temporary in-memory workspace
- File inventory: count, types, folder structure mapped and shown to user
- Each file routed by type (code → Reviewer, images → Design Agent, docs → Document Agent)
- Project Memory populated with full project context
- Clarification Agent asks: "What would you like to do with this project?"
- Options: Review entire codebase | Add a feature | Fix a bug | Rebuild | Analyse only
- Selected action → normal pipeline runs with full project context
- Output: modified ZIP with same folder structure + changelog

ZIP size limit: 100 MB. If the extracted content exceeds 500 files, Thinker Core summarises the structure and asks the user to select which subfolder or module to focus on first.

22.4 Code File Upload Flow

| Step                   | What Happens                                                                                                                                 |
|------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| 1. Upload              | Code file arrives. Thinker Core identifies language from extension and content.                                                              |
| 2. Auto-review         | Reviewer Agent reads the code. Critic Agent independently looks for issues. Both run automatically — no user prompt needed.                  |
| 3. Findings shown      | User sees: language detected, file size, issues found (by severity), strengths noted.                                                        |
| 4. User chooses action | 'Escalate issues'   'Fix only critical issues'   'Explain what this code does'   'Add a feature to this code'   'Rewrite this in [language]' |
| 5. Pipeline runs       | Builder executes the chosen action. Judge approves before result is shown.                                                                   |
| 6. Output              | Modified code file + diff showing what changed + brief explanation of changes.                                                               |

Security rule: if code contains patterns matching known malware, cryptomining, data exfiltration, or shell injection, Thinker Core warns the user explicitly and does not build or improve the code. The warning names the pattern category but does not provide detail that would help weaponise it.

22.5 Image Upload & Editing Flow

Images can be uploaded for three distinct purposes. Thinker Core identifies the intent from the user's message alongside the upload:

| Operation                  | User Intent Signal                                                                                                                 | Design Agent Action                                     | Credits |
|----------------------------|------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------|---------|
| Analyse only               | 'What is in this image?' / no edit instructions                                                                                    | Vision model describes content, objects, colours, style | 2       |
| Edit / remove object       | 'Remove the background' / 'Delete the image on the left' / 'Make the sky blue' / 'Object detected, removed, area filled naturally' | Image inpainting API applied                            | 15      |
| Style transfer             | 'Make this cartoon style' / 'Convert to sketch' / 'Simplify image to full image'                                                   | Style transfer API applied                              | 15      |
| Reference-based generation | 'Use this logo's style to make a banner' / 'Style this image like reference image. New asset generated using extracted style'      | Style extraction and generation API applied             | 20      |
| Upscale / enhance          | 'Make this higher resolution' / 'Sharpen this image'                                                                               | Upscaling API applied                                   | 8       |

All image edits follow the Visual Asset Approval Flow (Section 5.11): user sees result and can approve, revise, or regenerate. Regeneration capped at 3 attempts per edit operation.

22.6 Document Upload Flow (PDF, DOCX, TXT)

| User Intent                     | What Happens                                                                                              | Output                                             |
|---------------------------------|-----------------------------------------------------------------------------------------------------------|----------------------------------------------------|
| Summarise                       | Document Agent extracts key points, sections, conclusions                                                 | Structured summary in user's language              |
| Analyse / critique              | Document Agent reads for logic, gaps, contradictions                                                      | Strengths and weaknesses, specific recommendations |
| Extract data                    | Document Agent identifies tables, numbers, lists, names                                                   | Structured data export (JSON or table)             |
| Answer questions about document | Document added to context. User asks questions.                                                           | Direct answers with page/section references        |
| Use as project input            | Document becomes Clarification input — Thinker builds alongside user with document as requirements source |                                                    |
| Translate                       | Content extracted, translated to user's target language                                                   | Translated document in same format                 |

22.7 Document Agent — 13th Agent Role ★ NEW

Document Agent is a new agent role added to handle file-based input. It is the 13th agent in V1, joining the original twelve.

| Field              | Specification                                                                                                                            |
|--------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| Role               | Read, parse, and extract structured content from any document file. Convert unstructured document content into structured content.       |
| Input              | Uploaded file (PDF, DOCX, TXT, MD, CSV, XLSX) + user intent + conversation context                                                       |
| Output             | Structured content object: {summary, key_points, sections[], data_tables[], entities[], language_detected, page_count}                   |
| V1 Implementation  | Two-step: (1) file parsing library extracts raw text and structure; (2) single LLM call converts raw extraction into structured content. |
| Model Tier         | Fast/cheap — document parsing is mostly rule-based; LLM call is for structuring, not reasoning.                                          |
| next_action output | Same as all agents: proceed   clarify   retry   halt                                                                                     |
| Does NOT do        | Does not generate new content, write code, or make decisions. It reads and structures — other agents act on it.                          |

### 22.8 File Security Rules

**Executable files blocked:** Files with extensions .exe, .bat, .sh, .dll, .bin, .dmg, .apk, .ipa are rejected at upload. No credits charged. User told why.

**No file stored permanently:** Uploaded files live in temporary in-memory workspace for the duration of the session. They are deleted when the session ends or the pipeline completes. They are never written to persistent storage.

**File content stays in context only:** File content is sent to LLM providers as context in API calls — the same way a chat message is. It is not uploaded to a separate file storage service. The provider's data handling rules apply (same as any message content).

**Malicious code detection:** Code files are scanned for patterns matching: shell injection, data exfiltration, cryptomining, known malware signatures. Detection triggers a warning and blocks the build action. The scan runs before any LLM call.

**File size enforced pre-pipeline:** Size limits are checked before the file enters the pipeline. An oversized file is rejected immediately with a clear message — no credits are charged.

**No credentials in files:** If a code file contains what appears to be an API key, password, or token (detected by pattern matching), Thinker Core warns the user before proceeding and redacts the value from LLM context. Hard rule: credentials are never sent to model providers even as part of a code review.

### 22.9 File Upload Credit Cost Summary

| Operation                    | Credits | When Charged                               |
|------------------------------|---------|--------------------------------------------|
| Document analysis (per file) | 3       | When Document Agent completes successfully |
| Code review (per file)       | 5       | When Reviewer + Critic complete            |
| Code fix / improvement       | 10      | When Builder completes the fix             |
| ZIP extract + structure map  | 5       | When Thinker Core maps the structure       |
| Image analysis (vision)      | 2       | When Design Agent describes the image      |

| Operation                            | Credits                           | When Charged                            |
|--------------------------------------|-----------------------------------|-----------------------------------------|
| Image edit / object removal          | 15                                | When Design Agent returns edited image  |
| Image style transfer                 | 15                                | Per attempt                             |
| Reference-based image generation     | 20                                | Per attempt                             |
| Image upscale / enhance              | 8                                 | When upscaled image is returned         |
| Data file (CSV/XLSX) analysis        | 3                                 | When Research Agent structures the data |
| Full project from ZIP                | 5 (extract) + no multiple charges | Over pipeline agent                     |
| Blocked file (executable, oversized) | 0                                 | Never charged — rejected pre-pipeline   |



## 23. UI/UX Specification

This section defines every screen, panel, and interactive element in Thinker AI's user interface. It covers the main chat view, navigation, profile, settings, and all in-conversation controls. Design implementation follows the frontend-design skill conventions; this section defines what exists, not how it looks.

### 23.1 Overall App Layout

| Zone           | Location           | Contents                                                                                  |
|----------------|--------------------|-------------------------------------------------------------------------------------------|
| Left Sidebar   | Fixed, collapsible | Logo, Domain Picker, Conversation History, New Chat button, Upgrade CTA (Free/Pro u       |
| Main Chat Area | Centre, scrollable | Message thread, Thinking Level indicator, progress stages, output panels                  |
| Input Bar      | Bottom, fixed      | Text input, file upload button, Thinking Level selector, send button, credit balance chip |
| Right Panel    | Slide-in on demand | Project details, version history, pipeline log (founder mode), settings shortcuts         |
| Top Bar        | Fixed, minimal     | Current conversation title (editable), domain label, user avatar, notifications bell      |

### 23.2 Left Sidebar — Detail

| Domain Picker                            | Conversation History                           |
|------------------------------------------|------------------------------------------------|
| • CEO Agent (default — general judgment) | • Today / Yesterday / This Week / Older groups |
| • Coding                                 | • Search conversations (full-text)             |
| • DevOps                                 | • Pin a conversation to top                    |
| • Security                               | • Rename conversation                          |
| • Music                                  | • Delete conversation                          |
| • Design                                 | • Export conversation (PDF or TXT)             |
| • Data / Analytics                       | • Star / favourite a conversation              |
| • Marketing                              | • Project badge (if conversation has a build)  |
| • Finance                                | • Last message preview                         |
| • Legal                                  | • Credit spend shown per conversation          |
| • Research                               |                                                |
| • Education                              |                                                |
| • Healthcare                             |                                                |
| • + Custom domain (Pro/Founder only)     |                                                |

### 23.3 Input Bar — Detail

## Input Bar Controls

- Text input field — multiline, auto-expands, supports paste of code/URLs
- File upload button (paperclip icon) — opens file picker, supports drag-and-drop
- Thinking Level selector — tap to cycle , or long-press for picker
- Send button — disabled until input is non-empty or file is attached
- Credit cost preview chip — shows estimated credits before sending (updates as Thinking Level changes)
- Voice input button (V2 — not in V1 scope)
- Emoji / reaction button (V2)

## 23.4 Chat Message Types

| Message Type               | Visual Treatment                         | Contents                                                                                                           |
|----------------------------|------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| User message               | Right-aligned, user colour               | Text, attached file name/thumbnail                                                                                 |
| Thinker direct answer      | Left-aligned, clean                      | Text response, formatted markdown, code blocks                                                                     |
| Thinking Level indicator   | Inline badge before response             |  icon + level name + credit cost |
| Stage 1 — Thinking Summary | Expandable card                          | Goal understood, strategy brief, proposed approach, Confirm/Edit buttons                                           |
| Stage 2 — Blueprint        | Expandable card with checklist           | Step list with checkboxes, tech stack, Approve/Edit buttons                                                        |
| Stage 3 — Build Progress   | Split view: progress left, preview right | Live weight ticker + rendered UI preview, first-screen approval prompt                                             |
| Stage 4 — Approval         | Full-width card with 3 CTAs              | Looks good / Change something / Go back to blueprint                                                               |
| Stage 5 — Final Output     | Delivery card                            | Summary, Live Preview link, ZIP download, Architecture Notes                                                       |
| Clarification questions    | Distinct question card                   | Numbered questions, input fields inline, Submit button                                                             |
| Credit confirmation        | Modal overlay                            | Action name, credit cost, balance before/after, Confirm/Cancel                                                     |
| Upgrade prompt             | Inline banner                            | What was gated, which plan unlocks it, Upgrade CTA                                                                 |
| Error / halt message       | Amber card                               | What happened, what is saved, options to continue                                                                  |
| File received card         | Left-aligned, file icon                  | File name, type, size, detected language/content type, available actions                                           |
| Image output card          | Left-aligned, image preview              | Generated/edited image, Approve/Revise/Regenerate buttons, attempt counter                                         |
| Version history card       | Slide-in panel                           | Version list with timestamps, Restore button per version, diff preview                                             |

## 23.5 Profile Page

Accessed via the user avatar in the top bar. Contains all personal account information.

| Profile — Identity                                   | Profile — Activity                     |
|------------------------------------------------------|----------------------------------------|
| • Display name (editable)                            | • Total projects built (count)         |
| • Email address (editable, requires re-verification) | • Total Think Credits spent (all time) |
| • Profile photo (upload, crop, remove)               | • Favourite domains used               |
| • Username / handle (unique, used in shared links)   | • Most used Thinking Level             |
| • Bio / description (optional, 160 chars)            | • Total conversations                  |
| • Preferred language (overrides auto-detect)         | • Total files uploaded                 |
| • Date joined                                        | • Projects shared publicly             |
| • Account type badge (Free / Pro / Founder)          | • Member since date                    |

| Profile — Decision Memory                             | Profile — Linked Accounts        |
|-------------------------------------------------------|----------------------------------|
| • View all saved decisions (list)                     | • Google (for sign-in)           |
| • Edit a saved decision                               | • GitHub (for code import — V2)  |
| • Delete / revoke a decision                          | • Figma (for design import — V2) |
| • Add a decision manually                             | • Disconnect a linked account    |
| • Decision applies to: all projects / specific domain | • Add new linked account         |
| • Last applied: date shown per decision               |                                  |

## 23.6 Settings Page

Accessed via Settings in the left sidebar or top bar menu. Organised into eight tabs.

| Settings Tab: General                                                        |
|------------------------------------------------------------------------------|
| • App language (overrides auto-detect for UI elements)                       |
| • Response language (same as user language / always English / ask each time) |
| • Default Thinking Level (auto / always Low / always Medium / always High)   |
| • Default domain (none / pick one)                                           |
| • Code comment language (English / user language / ask each time)            |
| • Date & time format                                                         |
| • Timezone                                                                   |

- App theme (Light / Dark / System)

- Font size (Small / Medium / Large)

### Settings Tab: Thinking & Pipeline

- Auto-select Thinking Level (on/off)

- Show credit cost before sending (on/off — default on)

- Show pipeline progress stages (on/off)

- Show Thinking Level badge on every response (on/off)

- Signature Question (on/off — default on, can be disabled on Pro/Founder)

- Goal Discovery depth (auto / always full / always light)

- Strategy Agent (on/off per domain)

- Founder Mode auto-detect (on/off)

- Multi-language in pipeline (on/off — default on)

### Settings Tab: Memory

- Project Memory (on/off)

- Long-term Preference Memory (on/off)

- Decision Memory (on/off)

- Clear all Project Memory

- Clear all Long-term Memory

- Clear all Decision Memory

- Export all memory (JSON)

- Memory retention period (30 / 90 / 180 / forever)

### Settings Tab: Files & Uploads

- Default action on code upload (auto-review / ask each time)

- Default action on document upload (summarise / ask each time)

- Default action on image upload (analyse / ask each time)

- Default action on ZIP upload (map & ask / full review)

- Auto-scan code for security issues (on/off — default on)
- Warn before sending file content to LLM providers (on/off)
- Max file size limit (use platform default / set custom lower limit)

#### Settings Tab: Credits & Billing

- Current plan name + renewal date
- Think Credits balance (remaining this month)
- Credits used this month (with breakdown by action type)
- Buy extra credits button
- Upgrade / downgrade plan
- View billing history (invoice list)
- Download invoice (per month)
- Cancel subscription
- Low balance alert threshold (set custom — default 20 credits)
- Auto top-up (on/off — buy X credits when balance drops below Y)
- Payment method (view / update)

#### Settings Tab: Notifications

- Pipeline complete (push / email / none)
- Build failed / halted (push / email / none)
- Low credit balance alert (push / email / none)
- New feature announcements (email / none)
- Weekly usage summary (email / none)
- Billing / payment notifications (email — always on, cannot disable)
- Upgrade reminders (on/off)

#### Settings Tab: Privacy & Security

- Change password
- Two-factor authentication (on/off, TOTP or SMS)

- Active sessions (list, revoke any session)
- Data export (full account data as JSON)
- Delete account (with confirmation flow)
- Opt out of usage analytics (on/off)
- View data processing information
- API key management (Founder plan — view, create, revoke)

#### Settings Tab: Advanced (Founder plan only)

- API key creation and management
- Webhook URL (for build-complete notifications)
- Custom domain for Live Preview (V2)
- Pipeline log access (full routing\_history and failover\_log)
- Pool health dashboard (which models are currently active)
- Credit usage API (programmatic access to usage data)
- Custom Decision Memory rules via JSON import
- Priority Queue position visibility

## 23.7 In-Chat Controls (Per Message)

Every Thinker response message has a row of action icons beneath it.

| Control           | Icon          | Action                                                                  |
|-------------------|---------------|-------------------------------------------------------------------------|
| Copy              | Copy icon     | Copy full response text to clipboard                                    |
| Regenerate        | Refresh icon  | Re-run the same request at the same Thinking Level (costs same credits) |
| Thinking Level up | Arrow-up icon | Re-run at one level higher (costs difference in credits)                |
| Share             | Share icon    | Generate a shareable link to this response (public read-only)           |
| Thumbs up         | ■             | Positive feedback sent to Anthropic (no credits)                        |
| Thumbs down       | ■             | Negative feedback — opens feedback form, sent to Anthropic              |
| Pin               | Pin icon      | Pin this response to the top of the conversation                        |
| Save to project   | Folder icon   | Save this response as a named note in Project Memory                    |
| Version history   | Clock icon    | Open version history panel (project outputs only)                       |
| Rollback to this  | Undo icon     | Restore the project to the state at this output (project outputs only)  |

## 23.8 Right Panel — Project Details

Slides in when a project-type pipeline is active or complete.

| Project Tab                                     | Pipeline Log Tab (Founder plan)            |
|-------------------------------------------------|--------------------------------------------|
| • Project name (editable)                       | • Per-agent log entries (Section 9.2)      |
| • Created date                                  | • routing_history trace                    |
| • Last modified date                            | • failover_log (which models were swapped) |
| • Domain used                                   | • loop_counts per agent                    |
| • Thinking Level used                           | • Token cost breakdown per agent           |
| • Tech stack summary                            | • Failover cost vs normal cost             |
| • Credits spent on this project                 | • Total session duration                   |
| • Current status (building / complete / halted) | • Model pool health at session start       |
| • Live Preview link (if web project)            | • Export log as JSON                       |
| • Download ZIP button                           |                                            |
| • Delete project                                |                                            |

| Version History Tab                | Files Tab                            |
|------------------------------------|--------------------------------------|
| • List of versions (v1, v2, v3...) | • All files uploaded in this project |

|                                                |                                                             |
|------------------------------------------------|-------------------------------------------------------------|
| • Timestamp per version                        | • File name, type, size, upload date                        |
| • What changed (auto-generated diff summary)   | • Re-use file in new message                                |
| • Restore to this version button               | • Delete file from project context                          |
| • Compare two versions side-by-side (V2)       | • Files auto-deleted on session end if not saved to project |
| • Max 10 versions stored (older auto-deleted)  |                                                             |
| • Rollback note: restoring creates new version |                                                             |

## 23.9 Onboarding Flow (New User)

| Step | Screen               | What Happens                                                                                   |
|------|----------------------|------------------------------------------------------------------------------------------------|
| 1    | Welcome              | Brief explanation of what Thinker AI does. One sentence: 'Thinker AI thinks before it builds.' |
| 2    | Choose your plan     | Free Trial / Pro / Founder cards with feature comparison. Default: Free Trial.                 |
| 3    | First question       | Thinker asks: 'What are you here to build or figure out?' — no form, just a chat prompt.       |
| 4    | Thinking Level intro | After first message, a tooltip explains the four Thinking Levels. Dismissable.                 |
| 5    | Credit intro         | After first credit is spent, a tooltip explains Think Credits. Dismissable.                    |
| 6    | Domain Picker intro  | After second conversation, tooltip points to the Domain Picker. Dismissable.                   |
| 7    | Done                 | No more tooltips. User is in the product. Help available via '?' button in top bar.            |

## 23.10 Empty States & Zero-Data Screens

| Screen                          | Empty State Message                                                                |
|---------------------------------|------------------------------------------------------------------------------------|
| New conversation                | "What do you want to build, figure out, or create today?"                          |
| Conversation history (none yet) | "Your conversations will appear here."                                             |
| Version history (no versions)   | "No versions saved yet — versions appear here after your first build."             |
| Decision Memory (none set)      | "No saved decisions yet. Say 'always use React' and I'll remember it."             |
| Files tab (no files)            | "No files in this project yet."                                                    |
| Notifications (none)            | "You're all caught up."                                                            |
| Billing history (no invoices)   | "No invoices yet — your first invoice will appear after your first billing cycle." |